

说明

CH32V467 是南京沁恒微电子基于青稞 RISC-V3V 内核设计的工业级互联型微控制器，系统主频高达 200MHz。该芯片在 CH32V407 的基础上，额外内置了 4MB/8MB 的片内扩展 PSRAM 存储器，为数据密集型应用提供了充足的存储资源。

PSRAM（Pseudo Static RAM）是一种伪静态随机存储器，相较于传统 SDRAM，PSRAM 功耗更低，同时比 SRAM 具有更高的存储密度和更优的成本效益。CH32V467 内置的 PSRAM 通过芯片内部总线直接与内核和 DMA 控制器相连，支持高达 600MB/s 的数据传输速率，非常适合图形显示、音频处理、数据采集等大容量数据缓存场景。

本手册旨在帮助开发人员快速掌握 CH32V467 内置 PSRAM 的功能特性、配置方法和典型应用。

适用范围

适用范围	系列
通用 MCU	CH32V467

目录

说明	1
目录	2
图片索引	2
第 1 章 PSRAM 功能特性	1
1. 1 PSRAM 基本框架原理	1
1. 1. 1 PSRAM 电源	1
1. 1. 2 PSRAM 时钟	2
1. 2 PSRAM 特性	3
第 2 章 配置方法	4
2. 1 PSRAM 初始化配置	4
2. 1. 1 PSRAM 时钟初始化	4
2. 1. 2 PSRAM 控制器配置	4
2. 1. 3 PSRAM 写 Latency 配置	6
2. 1. 4 PSRAM 读 Latency 配置	7
2. 2 PSRAM 低功耗	9
第 3 章 读写应用	10
3. 1 CPU 读写	10
3. 2 专用 DMA 读写	10
3. 3 问题查找	12
历史版本	13
声明	14

图片索引

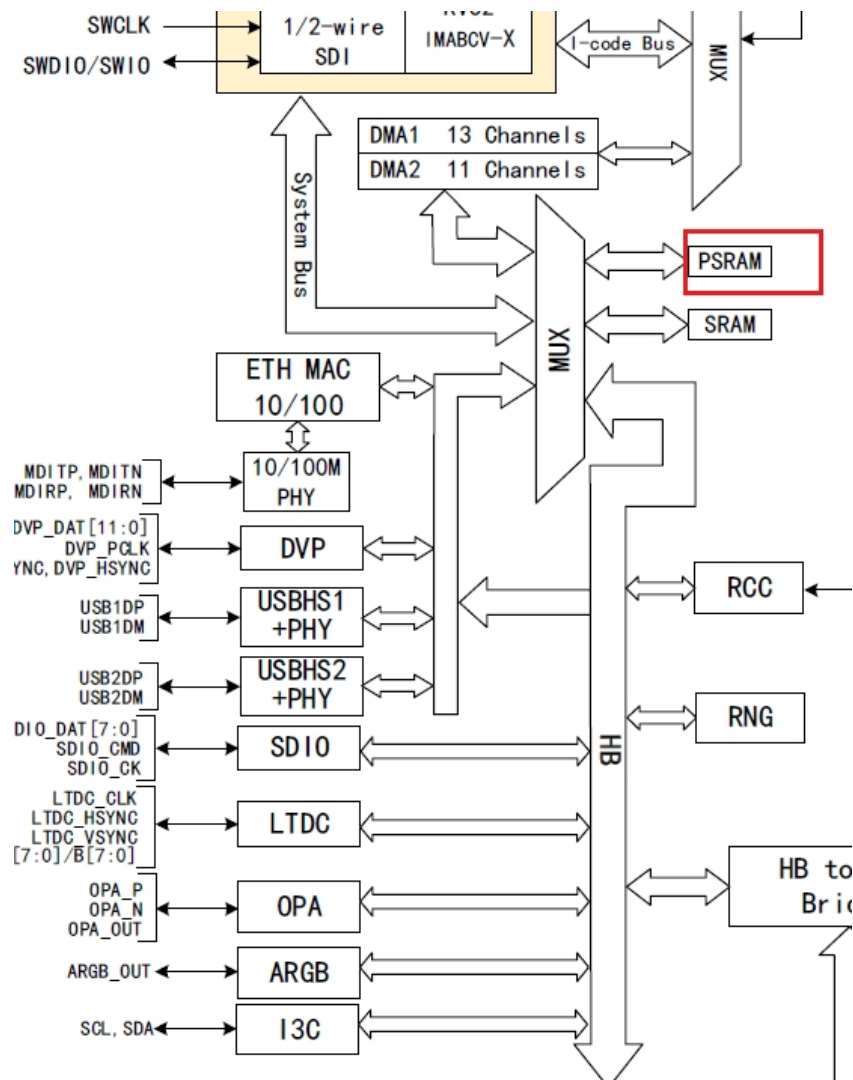
PSRAM 框架	1
PSRAM 电源框架	2
VDD18 框架	2
PSRAM 时钟来源	3
PSRAM 时钟配置	4
PSRAM 读写周期范围	5
PSRAM 连续突发间隔值	5
PSRAM 退出睡眠模式时间范围	5
PSRAM MR2 寄存器的容量定义	5
PSRAM MR 寄存器 Write Latency 的定义	6
PSRAM MR4 寄存器的定义	6
PSRAM MR 寄存器 Read Latency 的定义	7
PSRAM MR0 寄存器的定义	8
PSRAM MR6 寄存器的定义	9

第1章 PSRAM 功能特性

1.1 PSRAM 基本框架原理

PSRAM 通过选择器，可以协调多个总线主机（如 CPU、DMA1、DMA2、DVP 和 USB）对包括 PSRAM 在内的存储器和外设的访问，PSRAM 的控制器挂载在 HB 时钟上。

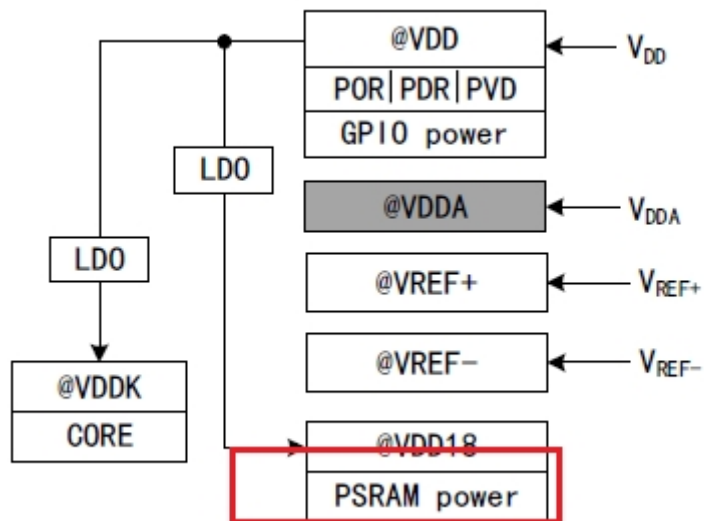
图 1-1 PSRAM 框架



1.1.1 PSRAM 电源

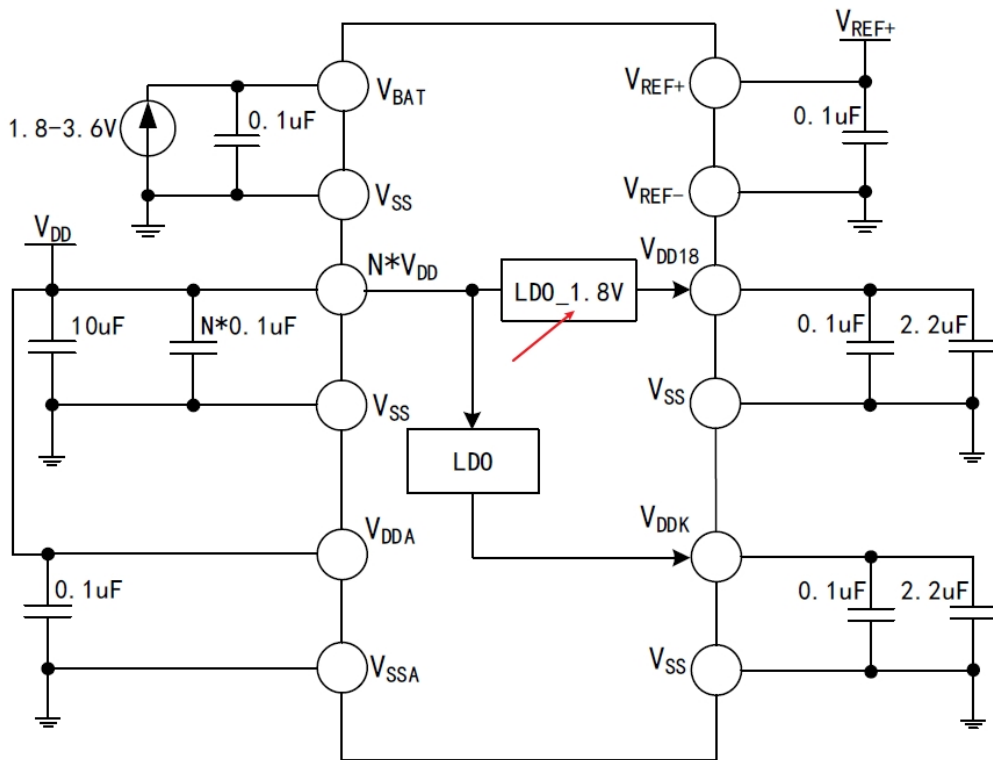
PSRAM 通过内部 VDD18 供电，典型值 1.8V，可以通过配置抬高至 1.9V，该电源可以通过外供 VDD，经过内部 LDO 降压后，专一供电给 PSRAM。也可以通过外供 VDD18 引脚提供。

图 1-2 PSRAM 电源框架



VDD18 引脚引出，输出电压，需外接 0.1uF 并联 2.2uF 容量的高频电容，MCU 上电运行后无需外供即可输出。也可上电运行后通过软件关闭内部 LDO，再外部直接供给 VDD18 引脚稳定的 1.8V 电压。

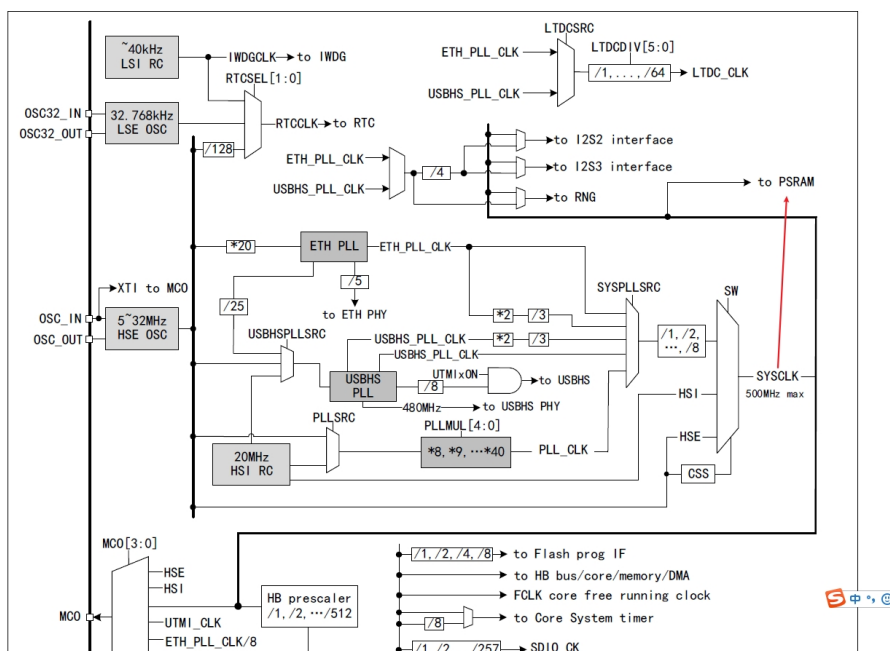
图 1-3 VDD18 框架



1.1.2 PSRAM 时钟

PSRAM 的工作时钟由 SYSCLK 提供，用户可通过配置系统时钟频率，使 PSRAM 接口达到最高 300MHz 的运行速率。

图 1-4 PSRAM 时钟来源



1.2 PSRAM 特性

内置的 PSRAM 特性:

- 1、容量：CH32V467RET6(4M x 8bits, 32Mb; 页面大小：1024 字节); CH32V467VET6 和 CH32V467WEU6 (8M x 8bits, 64Mb; 页面大小：1024 字节)
- 2、最大时钟频率: 300MHz, 最大数据速率: 600MBps
- 3、低功耗模式：部分阵列自刷新；自动温度补偿自刷新；可配置刷新率；保留数据的半睡眠模式
- 4、可软件复位：通过命令发送完成复位功能

第2章 配置方法

2.1 PSRAM 初始化配置

PSRAM 的初始化包括，控制时钟初始化，PSRAM 控制器初始化，PSRAM 的 Write Latency 配置和 Read Latency 配置。

2.1.1 PSRAM 时钟初始化

PSRAM 的时钟源于 SYSCLK，工程下的 main.c 文件中已经进行了配置。如图所示，代码的 SYSCLK 为 300MHz。

图 2-1 PSRAM 时钟配置

```

*****
* @fn      PSRAM_300MHz_HSE
*
* @brief    Sets PSRAM clock frequency to 300MHz.
*
* @return    none
*/
static void PSRAM_300MHz_HSE(void)
{
    __IO uint32_t StartUpCounter = 0, HSEStatus = 0, FLASH_Temp = 0;
    RCC->CTLR |= (uint32_t)0x00000001;
    RCC->CFGR0 &= (uint32_t)0x80FF0000;
    while ((RCC->CFGR0 & (uint32_t)RCC_SWS) != (uint32_t)0x00)
    {
    }

    RCC->CTLR &= (uint32_t)0xFE6FFFFF;
    RCC->CTLR &= (uint32_t)0xFFFBFFFF;

    RCC->CFGR0 &= (uint32_t)0xFFC0FFFF;
    RCC->CFGR2 &= (uint32_t)0xFFFFEFFF;
    RCC->CFGR2 &= (uint32_t)0x03F11800;

    RCC->CFGR0 |= (uint32_t)(0x20000000);
    RCC->CFGR3 = 0x00000000;
    RCC->INTR = 0x008F0000;
    RCC->CTLR |= ((uint32_t)RCC_HSEON);

    /* Wait till HSE is ready and if Time out is reached exit */
    do
    {
        HSEStatus = RCC->CTLR & RCC_HSERDY;
        StartUpCounter++;
    } while((HSEStatus == 0) && (StartUpCounter != HSE_STARTUP_TIMEOUT));

    if ((RCC->CTLR & RCC_HSERDY) != RESET)
    {
        HSEStatus = (uint32_t)0x01;
    }
    else

```

2.1.2 PSRAM 控制器配置

PSRAM 控制器的初始化首先要使能 HB 线上的 PSRAM 控制器时钟，开启时钟后，开始配置 PSRAM 相关寄存器。

PSRAM_trc 为读写周期的最小时间配置值，实际 $T_{rc} = \text{PSRAM_trc} \times \text{PSRAM 时钟的周期}$ ，必须要满足 Write / Read Cycle $T_{rc} \geq 60\text{ns}$

图 2-2 PSRAM 读写周期范围

符号 ^①	参数 ^②	条件 ^③										单位 ^④
		175MHz ^⑤		200MHz ^⑤		225MHz ⁽¹⁾ ⑤		250MHz ⁽¹⁾ ⑤		300MHz ⁽¹⁾ ⑤		
		MIN	MAX	MIN	MAX	MIN	MAX	MIN	MAX	MIN	MAX	
t _{SYSCLK}	系统时钟周期 ^⑥	6 ^⑦	⑧	5 ^⑦	⑧	4.4 ^⑦	⑧	4 ^⑦	⑧	3.3 ^⑦	⑧	ns ^⑨
t _{CEH}	非睡眠模式下, PSRAM_CE_n 低脉冲宽度 (CELPW[11:0]) ^⑥	⑧	4 ^⑦	⑧	4 ^⑦	⑧	4 ^⑦	⑧	4 ^⑦	⑧	4 ^⑦	us ^⑨
t _{RO}	PSRAM 读写周期 (TRC[7:0]) ^⑥	3 ^⑦	⑧	3 ^⑦	⑧	3 ^⑦	⑧	3 ^⑦	⑧	3 ^⑦	⑧	ns ^⑨
t _{HS}	PSRAM 半睡眠模式最短持续时间 ^⑥	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	ns ^⑨
t _{XHS}	PSRAM 半睡眠模式最短持续时间 ^⑥	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	us ^⑨
t _{XHS}	PSRAM 退出半睡眠模式的时间 ^⑥	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	us ^⑨
t _{XPHS}	退出半睡眠模式时, PSRAM_CE_n 的低脉冲宽度 (TXLPD[7:0]) ^⑥	60 ^⑦	2000 ^⑦	60 ^⑦	2000 ^⑦	60 ^⑦	2000 ^⑦	60 ^⑦	2000 ^⑦	60 ^⑦	2000 ^⑦	ns ^⑨
t _{DPD}	PSRAM 深睡眠模式最短持续时间 ^⑥	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	us ^⑨
t _{DPD}	PSRAM 深睡眠模式下的最小周期 ^⑥	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	500 ^⑦	⑧	us ^⑨
t _{XDPD}	PSRAM 退出深睡眠模式的时间 ^⑥	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	150 ^⑦	⑧	us ^⑨
t _{XDPD}	退出深睡眠模式时, PSRAM_CE_n 的低脉冲	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	60 ^⑦	⑧	ns ^⑨

PSRAM_t_{cp} 为两个连续突发之间的最小间隔配置值，实际 t_{CPH} = PSRAM_t_{cp} * PSRAM 时钟的周期，需满足图中最小值。PSRAM 连续突发间隔值

图 2-3 PSRAM 连续突发间隔值

Symbol	Parameter	BGA 1.8V Only									
		166MHz		200MHz		225MHz		250MHz		300MHz	
		Min	Max	Min	Max	Min	Max	Min	Max	Min	Max
t _{CPH}	CE# HIGH between subsequent burst operations	22		24		26		28		30	

PSRAM_t_{xl}pd 为退出低功耗时（半睡眠和深睡眠）PSRAM_CE_n 持续的脉冲宽度，t_{xphs}/t_{xdpd}=PSRAM_t_{xl}pd* HSI 时钟的周期，需满足图中的极值需求。

图 2-4 PSRAM 退出睡眠模式时间范围

符号 [Ⓐ]	参数 [Ⓐ]	条件 [Ⓐ]										单位 [Ⓐ]
		175MHz [Ⓐ]		200MHz [Ⓐ]		225MHz ^{(1) Ⓐ}		250MHz ^{(1) Ⓐ}		300MHz ^{(1) Ⓐ}		
		MIN [Ⓐ]	MAX [Ⓐ]	MIN [Ⓐ]	MAX [Ⓐ]	MIN [Ⓐ]	MAX [Ⓐ]	MIN [Ⓐ]	MAX [Ⓐ]	MIN [Ⓐ]	MAX [Ⓐ]	
t _{XPHS}	退出半睡眠模式时， PSRAM_CE_n 的低脉冲 宽度（TXLPD[7:0]） [Ⓐ]	60 [Ⓐ]	2000 [Ⓐ]	60 [Ⓐ]	2000 [Ⓐ]	60 [Ⓐ]	2000 [Ⓐ]	60 [Ⓐ]	2000 [Ⓐ]	60 [Ⓐ]	2000 [Ⓐ]	ns [Ⓐ]
t _{XDPD} [Ⓐ]	退出深睡眠模式时， PSRAM_CE_n 的低脉冲	60 [Ⓐ]	[Ⓐ]	60 [Ⓐ]	[Ⓐ]	60 [Ⓐ]	[Ⓐ]	60 [Ⓐ]	[Ⓐ]	60 [Ⓐ]	[Ⓐ]	ns [Ⓐ]

PSRAM_cap_cfg 为 PSRAM 的容量配置，该值需要根据芯片的型号来区分，一般设置为 32Mb 或者 64Mb。该值可以通过读取 MR2 寄存器来获取

图 2-5 PSRAM MR2 寄存器的容量定义

Table 9- MR2 Mode register Definition

Field	Bits	Description
Device Density mapping	OP[2:0]	001B 32Mb(default)
		011B 64Mb
		101B 128Mb
		111B 256Mb
		others reserved

PSRAM_cfifo 为设置的 DMA 写操作清空 FIFO 方式选择，一般设置为:: 写地址和读 buf 中的地址匹配才把读 FIFO 清空。

PSRAM_exti_lpmdd 表示 PSRAM 退出当前低功耗状态使能，当初始化的时候应该配置为退出低功耗模式，相关寄存器配置完成后，延时等待初始化完成

2.1.3 PSRAM 写 Latency 配置

Write Latency 配置主要使用到 MR4 寄存器的[5:7]bit。

图 2-6 PSRAM MR 寄存器 Write Latency 的定义

MR num	MA[7:0]	READ/ WRITE	OP7	OP6	OP5	OP4	OP3	OP2	OP1	OP0
0	h00	R/W	TSO	Drive Str ext	LT	Read Latency Code			Drive Str.	
1	h01	R	ULP	RFU		Vendor ID				
2	h02	R	KGD			Dev ID		Density		
3	h03	R	0	0	SRF	RFU		RFU		
4	h04	R/W	Write Latency Code			RF		PASR		
6	h06	W	Half Sleep							

根据配置的不同的 PSRAM 时钟，设置 MR4 对应的值。配置参数需小于等于 300Mhz。

图 2-7 PSRAM MR4 寄存器的定义

Field	Bits	Description																																				
PASR	OP[2:0]	If PASR (Partial Array Self-Refresh) is enabled, only data located in the specified areas of the array will be refreshed. This feature can reduce standby current.																																				
		address space RA[11:0].CA[9:0]																																				
		Codes	Refresh Coverage	Address Space	Size	Density																																
		000B	Full array (default)	000000h-3FFFFFFh	4M x8	32Mb																																
		001B	Bottom 1/2 array	000000h-1FFFFFFh	2M x8	16Mb																																
		010B	Bottom 1/4 array	000000h-0FFFFFFh	1M x8	8Mb																																
		011B	Bottom 1/8 array	000000h-07FFFFFFh	512K x8	4Mb																																
		100B	None	0	0M	0Mb																																
		101B	Top 1/2 array	200000h-3FFFFFFh	2M x8	16Mb																																
		110B	Top 1/4 array	300000h-3FFFFFFh	1M x8	8Mb																																
111B	Top 1/8 array	380000h-3FFFFFFh	512K x8	4Mb																																		
Refresh Frequency	OP[4:3]	x0B Always 4x Refresh(default) 01B Enables 1x Refresh when temperature allows 11B Enable 0.5x Refresh when temperature allows Note: x = don't care																																				
Write Latency	OP[7:5]	Write latency, WLC, default to 5 after power up Write Latency Code contain MR4[7:5] and MR8[5]																																				
		<table><tr><th>MR8[5]</th><th>MR4[7:5]</th><th>Write Latency</th><th>Fmax (MHz)</th></tr><tr><td>0B</td><td>000B</td><td>3</td><td>66</td></tr><tr><td>0B</td><td>100B</td><td>4</td><td>109</td></tr><tr><td>0B</td><td>010B</td><td>5(default)</td><td>133</td></tr><tr><td>0B</td><td>110B</td><td>6</td><td>166</td></tr><tr><td>0B</td><td>001B</td><td>7</td><td>200</td></tr><tr><td>0B</td><td>101B</td><td>8</td><td>225</td></tr><tr><td>0B</td><td>011B</td><td>9</td><td>250</td></tr><tr><td>0B</td><td>111B</td><td>11</td><td>300</td></tr></table>	MR8[5]	MR4[7:5]	Write Latency	Fmax (MHz)	0B	000B	3	66	0B	100B	4	109	0B	010B	5(default)	133	0B	110B	6	166	0B	001B	7	200	0B	101B	8	225	0B	011B	9	250	0B	111B	11	300
MR8[5]	MR4[7:5]	Write Latency	Fmax (MHz)																																			
0B	000B	3	66																																			
0B	100B	4	109																																			
0B	010B	5(default)	133																																			
0B	110B	6	166																																			
0B	001B	7	200																																			
0B	101B	8	225																																			
0B	011B	9	250																																			
0B	111B	11	300																																			

设置对应库函数，赋值 PSRAM_Fre 和对应的 Write_Latency

```

/*****

```

```

* @fn      PSRAMSetWrLatency

```

```

*

```

```

* @brief   PSRAM set write Latency

```

```

*

```

```

* @param   PSRAM_Fre - MR4_Write_66M

```

```

                MR4_Write_109M

```

```

                MR4_Write_133M

```

```

                MR4_Write_166M

```

```

                MR4_Write_200M

```

```

                MR4_Write_225M

```

```

                MR4_Write_250M

```

```

                MR4_Write_300M

```

```

                MR4_Write_333M

```

```

                MR4_Write_400M

```

```

                Write_Latency - Latency_66M

```

```

                Latency_109M

```



```

Latency_133M
Latency_166M
Latency_200M
Latency_225M
Latency_250M
Latency_300M
Latency_333M
Latency_400M

* @return none
*/ void SetWrLatency(uint32_t PSRAM_Fre, uint32_t Write_Latency)
{
    PSRAMSetData(PSRAM_Fre<<5);
    PSRAMSetWrLatency(Write_Latency);
    PSRAM_Set_MR_ADDR(CMD_MR_ADDR_4);
    PSRAM_Set_CMD(CMD_MR_WRITE);
    PSRAM_Set_Busy(ENABLE);
    PSRAM_Set_MW(ENABLE);
    Wait_Busy_0();

    if((PSRAM_Fre==MR4_Write_333M)|(PSRAM_Fre==MR4_Write_400M))
    {
        //High frequency latency enable for 333Mhz/400MHz .
        PSRAMSetData(Hfreq_En<<5);
        PSRAM_Set_MR_ADDR(CMD_MR_ADDR_8);
        PSRAM_Set_CMD(CMD_MR_WRITE);
        PSRAM_Set_Busy(ENABLE);
        PSRAM_Set_MW(ENABLE);
        Wait_Busy_0();
    }
}
```

2.1.4 PSRAM 读 Latency 配置

配置 Read Latency 需要对 MR0 寄存器的[2:4]bit。

图 2-8 PSRAM MR 寄存器 Read Latency 的定义

MR num	MA[7:0]	READ/ WRITE	OP7	OP6	OP5	OP4	OP3	OP2	OP1	OP0
0	'h00	R/W	TSO	Drive Str ext	LT	Read Latency Code			Drive Str.	
1	'h01	R	ULP	RFU			Vendor ID			
2	'h02	R	KGD			Dev ID		Density		
3	'h03	R	0	0	SRF	RFU				
4	'h04	R/W	Write Latency Code			RF		PASR		
6	'h06	W	Half Clean							

根据配置的不同的 PSRAM 时钟，设置 MR0 对应的值。配置参数需小于等于 300Mhz。

图 2-9 PSRAM MR0 寄存器的定义

Field	Bits	Description
Drive Str	OP[1:0]	Output Drive Strength codes. Combined with MR0[6].
		MR0[6] MR0[1:0] Write Latency
		0B 00B Full (25Ω default)
		0B 01B Half (50Ω)
		0B 10B 1/4 (100Ω)
		0B 11B 1/8 (200Ω)
		1B 00B 33Ω
		1B 11B 240Ω
		1B others reserved
Read Latency Code	OP[4:2]	Read latency is related with Read Latency Type(OP[5]). Read latency is pushed out up to LCx2 when internal Refresh is ongoing. Read Latency Code contain MR0[4:2] and MR8[5]

设置对应库函数，赋值 PSRAM_Fre 和对应的 Read_Latency。

```

/*****
* @fn      PSRAMSetRdLatency
*
* @brief   PSRAM set read Latency
*
* @param   PSRAM_Fre - MR0_Read_66M
*           MR0_Read_109M
*           MR0_Read_133M
*           MR0_Read_166M
*           MR0_Read_200M
*           MR0_Read_225M
*           MR0_Read_250M
*           MR0_Read_300M
*           MR0_Read_333M
*           MR0_Read_400M
*
*           Read_Latency - Latency_66M
*                           Latency_109M
*                           Latency_133M
*                           Latency_166M
*                           Latency_200M
*                           Latency_225M
*                           Latency_250M
*                           Latency_300M
*                           Latency_333M
*                           Latency_400M
*
* @return  none
*/
void SetRdLatency(uint32_t PSRAM_Fre, uint32_t Read_Latency,uint32_t Read_Latency_Type)
{

```

```
PSRAMSetData(PSRAM_Fre<<2);
PSRAMSetRdLatency(Read_Latency);
PSRAM_Set_MR_ADDR(CMD_MR_ADDR_0);
PSRAM_Set_CMD(CMD_MR_WRITE);
PSRAM_Set_Busy(ENABLE);
PSRAM_Set_MW(ENABLE);
Wait_Busy_0();
//High frequency latency enable for 333Mhz .
if((PSRAM_Fre==MR0_Read_333M)|(PSRAM_Fre==MR0_Read_400M))
{
    PSRAMSetData(Hfreq_En<<5);
    PSRAM_Set_MR_ADDR(CMD_MR_ADDR_8);
    PSRAM_Set_CMD(CMD_MR_WRITE);
    PSRAM_Set_Busy(ENABLE);
    PSRAM_Set_MW(ENABLE);
    Wait_Busy_0();
}
if(Read_Latency_Type==Read_Fixed)
{
    PSRAMSetData(Read_Fixed<<5);
    PSRAMSetRdLatency(Read_Latency);
    PSRAM_Set_MR_ADDR(CMD_MR_ADDR_0);
    PSRAM_Set_CMD(CMD_MR_WRITE);
    PSRAM_Set_Busy(ENABLE);
    PSRAM_Set_MW(CMD_MW_Write);
    Wait_Busy_0();
}
}
```

2.2 PSRAM 低功耗

PSRAM 可以配置进入半睡眠模式，通过对 MR6 寄存器进行写入 0XF0 的值后，可以进入半睡眠，该模式可以对数据进行保持，唤醒后数据依然存在。

图 2-10 PSRAM MR6 寄存器的定义

Table 12- MR6 Mode register Definition

Field	Bits	Description
HSM	OP[7:0]	11110000B Half Sleep mode others reserved

```
PSRAMWriteReg(CMD_MR_ADDR_6,0xF0 );//进入半睡眠模式
Delay_Us(150);
Delay_Us(50);
PSRAM->CTLR|=1<<16;//退出低功耗模式
Delay_Us(150);
Delay_Us(50);
```

第3章 读写应用

PSRAM 的读写可以通过直接 CPU 读写也可以通过专用 DMA 进行读写

3.1 CPU 读写

通过 CPU 直接读写 PSRAM，支持 8 位、16 位或 32 位随机或成块读写，经过测试，在时钟设置为 200MHz 的情况下，32 位写的速度为：130MB/s，读的速度为：111MB/s。由于测试时需要运行 for 循环代码，会损耗 CPU 的运行时间，所以测试出的速度会略低于理论值。

```
printf("CPU write data\r\n");
for(i=0; i<BUFFER_SIZE; i++)
{
    *(u32*)(PSRAM_ADDR + 4*i) = TxBuffer[i];
}

printf("CPU Read data\r\n");
for(i=0; i<BUFFER_SIZE; i++)
{
    RxBuffer[i]= *(vu32*)(PSRAM_ADDR + 4*i);
}
```

3.2 专用 DMA 读写

通过 PSRAM 专用 DMA 读写 PSRAM，经过测试，在时钟设置为 300MHz 的情况下，写的速度为：562MB/s，读的速度为：550MB/s。调用 PSRAMDMAWrite 和 PSRAMDMARead 函数，函数需要传递的为：存储写入和读取的 buff，写入或者读取的起始地址，数据宽度（可以选择 16 位或者 32 位），DMA 一次突发长度，两次突发之间暂停时间（这个参数值越小，连续读写的速度越快）。

```

/*****
* @fn      PSRAMDMAWrite
*
* @brief   PSRAM write data by DMA
*
* @param   pBuffer - data
*           WriteAddr - PSRAM address.
*           dataWidth - PSRAM_MEMORYSIZE_16bit:16bit
*                   PSRAM_MEMORYSIZE_32bit:32bit
*           ONEBL - DMA_BRST_NUM2: 2 times Burst Length
*                   DMA_BRST_NUM4: 4 times Burst Length
*                   DMA_BRST_NUM6: 6 times Burst Length
*                   DMA_BRST_NUM8: 8 times Burst Length
*                   DMA_BRST_NUM16:16 times Burst Length
*           TOWBL_TOUT- DMA_PAUSE_TIM0      pause 0 times
*                   DMA_PAUSE_TIM1      pause 1 times
*                   DMA_PAUSE_TIM2      pause 2 times
*                   DMA_PAUSE_TIM3      pause 3 times
*                   DMA_PAUSE_TIM4      pause 4 times
*                   DMA_PAUSE_TIM5      pause 5 times
*                   DMA_PAUSE_TIM6      pause 6 times
*                   DMA_PAUSE_TIM7      pause 7 times
*****/

```

```

DMA_PAUSE_TIM8      pause 8 times
DMA_PAUSE_TIM9      pause 9 times
DMA_PAUSE_TIM10     pause 10 times
DMA_PAUSE_TIM11     pause 11 times
DMA_PAUSE_TIM12     pause 12 times
DMA_PAUSE_TIM13     pause 13 times
DMA_PAUSE_TIM14     pause 14 times
DMA_PAUSE_TIM15     pause 15 times
*
* @return none
*/
Void PSRAMDMAWrite(uint32_t *pBuffer,uint32_t WriteAddr,uint32_t n,uint32_t dataWidth,uint32_t
ONEBL,uint32_t TOWBL_TOUT)
{
    PSRAMDMATypeDef PSRAMDMAstruct={0};
    PSRAMDMAstruct.PSRAM_DMA_DAT_DIR=DMA_DIR_PSRAM;
    PSRAMDMAstruct.PSRAM_DMA_DATA_NUM=n;
    PSRAMDMAstruct.PSRAM_DMA_DMA_ONEBL=ONEBL;
    PSRAMDMAstruct.PSRAM_DMA_MEMORY_SIZE=dataWidth;
    PSRAMDMAstruct.PSRAM_DMA_TOWBL_TOUT=TOWBL_TOUT;
    PSRAMDMASet(pBuffer,WriteAddr,&PSRAMDMAstruct);
    PSRAM_DMA_Cmd(ENABLE);
    while (((PSRAM->ISR)&0x0000002)==0 ){
    }
    PSRAM_ClearITPendingBit(PSRAM_DMATF);
    PSRAM_DMA_Cmd(DISABLE);
}

/*****
* @fn      PSRAMDMARead
*
* @brief    PSRAM write data by DMA
*
* @param    pBuffer - data
*           WriteAddr - PSRAM address.
*           dataWidth- Data bit width
*           ONEBL - DMA_BRST_NUM2: 2 times Burst Length
*                  DMA_BRST_NUM4: 4 times Burst Length
*                  DMA_BRST_NUM6: 6 times Burst Length
*                  DMA_BRST_NUM8: 8 times Burst Length
*                  DMA_BRST_NUM16:16 times Burst Length
*           TOWBL_TOUT- DMA_PAUSE_TIM0 pause 0 times
*                      DMA_PAUSE_TIM1 pause 1 times
*                      DMA_PAUSE_TIM2 pause 2 times
*                      DMA_PAUSE_TIM3 pause 3 times
*                      DMA_PAUSE_TIM4 pause 4 times
*****/

```

DMA_PAUSE_TIM5	pause 5 times
DMA_PAUSE_TIM6	pause 6 times
DMA_PAUSE_TIM7	pause 7 times
DMA_PAUSE_TIM8	pause 8 times
DMA_PAUSE_TIM9	pause 9 times
DMA_PAUSE_TIM10	pause 10 times
DMA_PAUSE_TIM11	pause 11 times
DMA_PAUSE_TIM12	pause 12 times
DMA_PAUSE_TIM13	pause 13 times
DMA_PAUSE_TIM14	pause 14 times
DMA_PAUSE_TIM15	pause 15 times
*	
* @return none	
*/	
void PSRAMDMARead(uint32_t *pBuffer,uint32_t ReadAddr,uint32_t n,uint32_t dataWidth,uint32_t	
ONEBL,uint32_t TOWBL_TOUT)	
{	
PSRAMDMATypeDef PSRAMDMAstruct={0};	
PSRAMDMAstruct.PSRAM_DMA_DAT_DIR=DMA_DIR_MEM;	
PSRAMDMAstruct.PSRAM_DMA_DATA_NUM=n;	
PSRAMDMAstruct.PSRAM_DMA_DMA_ONEBL=ONEBL;	
PSRAMDMAstruct.PSRAM_DMA_MEMORY_SIZE=dataWidth;	
PSRAMDMAstruct.PSRAM_DMA_TOWBL_TOUT=TOWBL_TOUT;	
PSRAMDMASet(pBuffer,ReadAddr,&PSRAMDMAstruct);	
PSRAM_DMA_Cmd(ENABLE);	
while (((PSRAM->ISR)&0x00000002)==0){};	
PSRAM_ClearITPendingBit(PSRAM_DMATF);	
PSRAM_DMA_Cmd(DISABLE);	
}	

3.3 问题查找

当 PSRAM 出现读写错误可以从一下方向查找问题根源：

- 1、检查芯片丝印，查看是否为 CH32V467，如果是 CH32V407，则无内置 PSRAM，需更换芯片。
- 2、测量 VDD18 引脚是否有稳定的 1.8V 电压输出。如果电压波动较大，则外供 1.8V 尝试。
- 3、软件复位后，退出低功耗再试，可能芯片误操作进入了低功耗模式。
- 4、重新查看初始化参数，是否与 PSRAM 时钟相匹配。

读取 PSRAM 内部参数，如：ID，容量等，如果有误，则判断为 PSRAM 时序有误，降低 PSRAM 时钟，重新配置读写 Latency 后查看是否可以正常读写。注意 PSRAM 初始化时 PSRAM 时钟需要大于 150MHz。

历史版本

日期	版本	变更内容
2026/06/25	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。